Create Our First AI Agent, for FREE!

Today, we will explore how to create AI agents for free.

But before we start building one, let's first take a deep dive into what agents actually are, how they work, and how they are different from LLMs.

Let's begin the journey.

1.1 LLM vs Agent

If we go back to earlier versions of GPT, like GPT-2 or GPT-3, and ask questions such as:

- What is the weather in Patna today?
- Plan a trip from Delhi to Dehradun. Compare bus, train, and flight fares, find a good stay within a moderate budget, and create a complete itinerary.
- Calculate 45678289291991 × 41201010188.

In these kinds of tasks, a standalone LLM would often fail.

But an agent can succeed.

Why is that?

How does an agent handle these complex tasks while a simple LLM struggles?

Let's discuss this in detail in the next section.

1.2 What are Agents?

Before understanding agents, let's first understand what LLMs do.

Large Language Models (LLMs) primarily predict the next token or word in a sequence. They generate text so fluently that it feels like real understanding, but technically, they are predicting the next most probable word based on patterns learned during training.

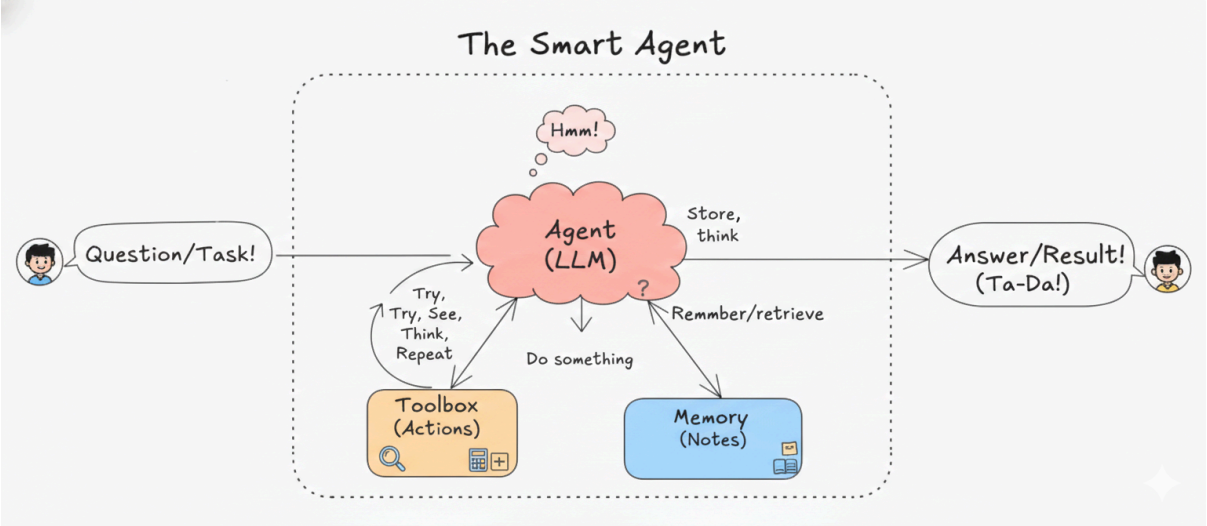
LLMs can generate responses, but they do not have autonomy. They cannot independently call tools or take actions.

Agents, on the other hand, are built on top of LLMs but have autonomy. You can think of tools as functions (for example, Python functions) that an agent can call when needed.

For example, if we ask an AI agent, "What is the weather in Roorkee today?", it can call a web search tool or a weather API, retrieve real-time information, and then present the result to us.

One-Line Definition

An agent is a software system that uses an LLM to pursue goals and complete tasks on behalf of users. It can reason, plan, call tools, and rely on memory to complete tasks.

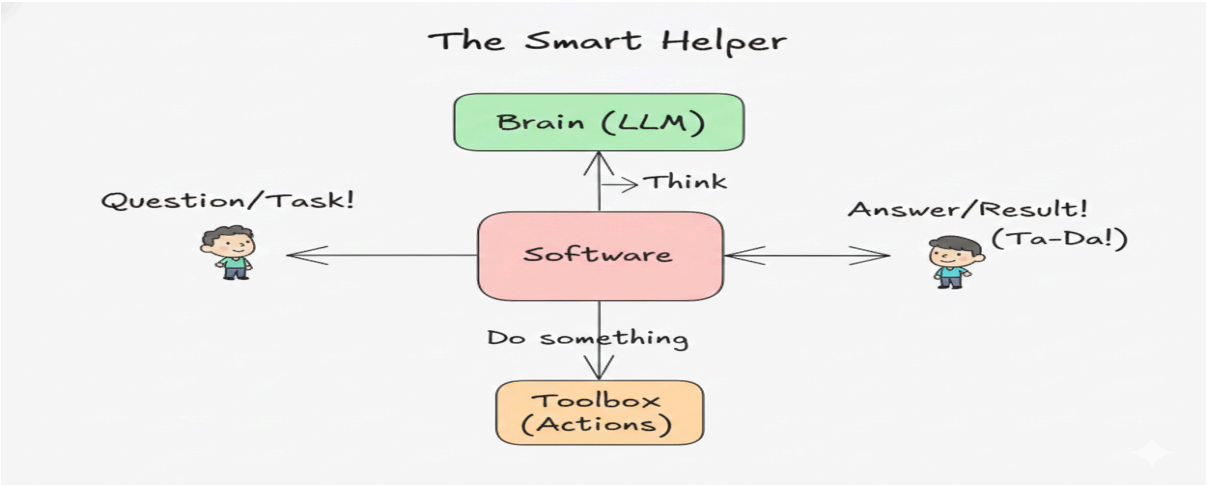


Want to Dive Deeper?

Explore workflow architectures and agent frameworks like ReAct, ReWOO, and Tree Search to better understand how agents reason and act.

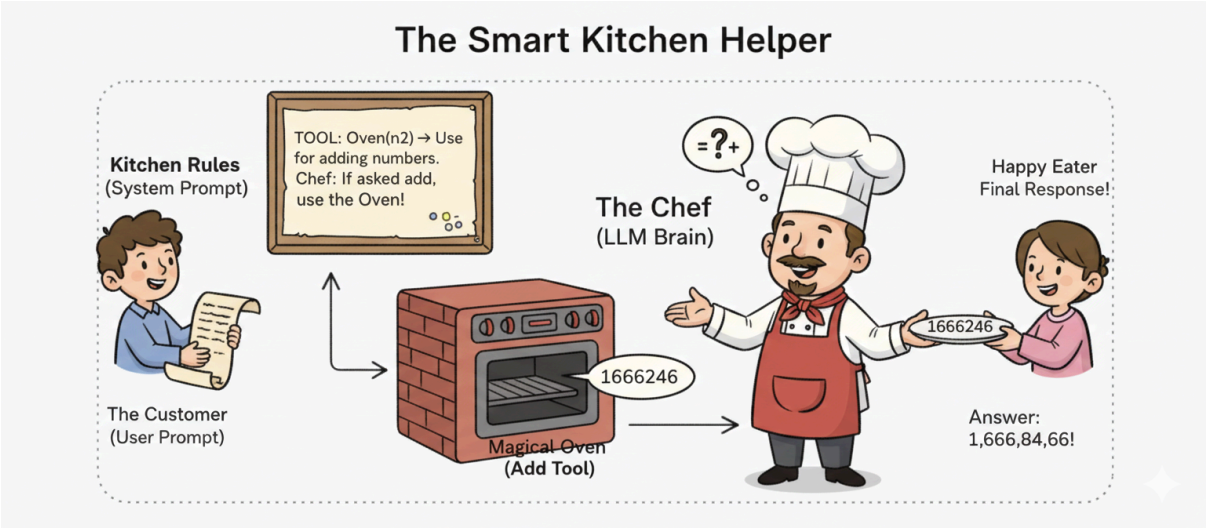
1.3 Tool Calling

Tools are like Python functions that an agent can call whenever needed.



First, we define the tool. Then, in the system prompt, we provide the function description. Whenever the LLM decides that the tool is required, it calls the tool and receives its response.

Let's visualize this process in the image below.



1.4 Creating Agents

Now we are ready to create agents.

There are many processes and frameworks available for building agents, such as LangChain, LangGraph, AutoGen, CrewAI, and Google ADK.

In this article, we will explore simple methods:

- A naive Python implementation

Using Ollama (For Free Local Testing)

So we will use **Ollama**. It is best for testing purposes.

We can download Ollama on Windows. Here we get small models that we can download and use locally. There is no API key limit, which makes it best for learning.

So we start with the **naive Python approach**.

In this approach, we build a simple agent.

Naive Python Approach

Step 1: Define the Tool

```python
def get_current_weather(city,unit="celsius"):
    return f"It is 23 {unit[0].upper()} and sunny in {city}"

get_current_weather("London")
```

Step 2: Let the Model Know About the Tools

```python
SYSTEM_PROMPT=(
    "You are an assistant that can call tools. "
    "When the user asks something requiring fresh data, respond **only** with a JSON like: "
    'TOOL_CALL:{"name": <tool_name>, "args": { ... }}.'
)

TOOL_SPEC="""
You can call exactly one tool:
- name: get_current_weather
  description: Return the current weather for a city.
  arguments:
    city: string
    unit: "celsius" | "fahrenheit"  (optional, default "celsius")
"""
USER_QUESTION = "What is the weather in San Diego today?"
```

I created a SYSTEM_PROMPT like this:

```
SYSTEM_PROMPT = (
    "You are an assistant that can call tools. "
    "When the user asks something requiring fresh data, respond only with JSON like: "
    'TOOL_CALL:{"name": <tool_name>, "args": { ... }}.'
)
```

Then I described the tool:

```
TOOL_SPEC = """
You can call exactly one tool:
- name: get_current_weather
  description: Return the current weather for a city.
  arguments:
    city: string
    unit: "celsius" | "fahrenheit"
"""
```

You think what's happening here?

I am manually teaching the LLM:

- What tool exists
- What it does
- How to call it
- What format to return

There is no automatic tool registration.

I pass everything manually in the prompt.

That's why it's called naive.

Step 3: Send the Query to the Model

```
from openai import OpenAI
client =OpenAI(api_key="ollama",base_url="http://localhost:11434/v1")

MODEL="gemma3:1b"

response=client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role":"system","content":SYSTEM_PROMPT+ "\n\n"+ TOOL_SPEC},
        {"role":"user","content":USER_QUESTION},
    ],
    temperature=0,
)

raw=response.choices[0].message.content
print("\nModel output:\n",raw)
```

Then I send:

messages = [
    {"role": "system", "content": SYSTEM_PROMPT + "\n\n" + TOOL_SPEC},
    {"role": "user", "content": USER_QUESTION},
]

Example question:

USER_QUESTION = "What is the weather in Patna today?"

Now the LLM decides:

- Should I answer directly?
- Or should I call the tool?

Step 4 & 5: Manually Detect and Execute the Tool Call

After getting the model response:

raw = response.choices[0].message.content

I manually check if it returned something like:

TOOL_CALL: {"name": "get_current_weather", "args": {...}}

Using regex:

pattern = r'TOOL_CALL:\s*({.*})'

match = re.search(pattern, raw, flags=re.DOTALL)

This is a very important learning:

There is no automatic tool execution.

I manually:

- Parse the model output
- Extract JSON
- Identify function name
- Extract arguments

This is pure Python control logic.

If a tool call is detected:

fn_name = call_json["name"]

args = call_json.get("args", {})

result = globals()[fn_name](**args)

Here's what's happening:

- `globals()[fn_name]` dynamically finds the function
- `(**args)` passes arguments
- The tool executes

Then I print the result.

Step 6: Modernising Tools Approach

```python
from pprint import pprint
import inspect


def to_schema(fn):
    sig = inspect.signature(fn)
    props = {
        name: {
            'type': 'number' if param.annotation in (int, float) else 'string',
            'description': f"Argument {name}"
        }
        for name, param in sig.parameters.items()
    }
    return {
        'name': fn.__name__,
        'description': (fn.__doc__ or '').strip().split('\n')[0],
        'parameters': {
            'type': 'object',
            'properties': props,
            'required': [n for n, p in sig.parameters.items() if p.default is inspect._empty]
        }
    }

tool_schema = to_schema(get_current_weather)
pprint(tool_schema)
```

Using inspect.signature() to automatically:

- Extract function parameters
- Create JSON schema
- Define required fields

This is still naive approach, but more structured.

Instead of manually writing tool descriptions,
I generated them dynamically from the function.

This is exactly how modern frameworks internally work.